

공세민

Se Min Kong

로봇과 소프트웨어를 연결하는 개발자

안녕하세요. 공세민입니다.

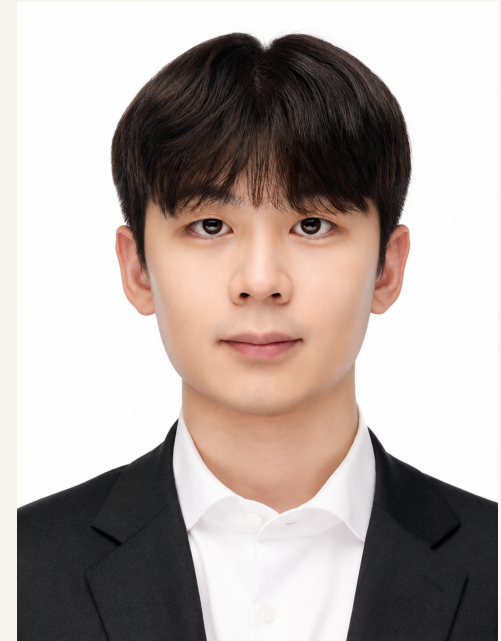
숭실대학교에서 소프트웨어를 전공했습니다.

현재 SSAFY Robotics Track에서 공부하고 있습니다.

센서 입력부터 제어 명령과 실제 동작까지 연결하며,
어떤 조건에서 동작하고 언제 오류가 나는지 확인합니다.

semin1224@gmail.com

github.com/SeMinKong



현재

SSAFY Robotics Track
2026.01 - 현재

학력

숭실대학교
소프트웨어학부
2020.03 - 2026.02

관심 분야

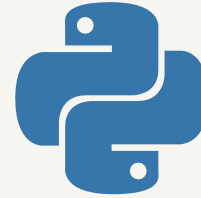
Computer Vision
Robotics / Physical AI

거주지

Suwon,
Republic of Korea

학습 스택

사용 언어



로봇·시뮬레이션



Isaac Sim



Isaac Lab

AI·에이전트



로컬 추론·API



개발 환경

프로젝트

장비 연동과 실물 제어를 중심으로, 입력과 출력의 품질을 다룬 작업을 소개합니다.

쪽	프로젝트	역할	핵심 판단
4 - 8	THING	6인 팀 / 구동·기구 통합	기구 편차 / 초기 목표와 토크 순서
9 - 14	AQIS	2인 팀 / 팀장·서버·장비 통합	검사·분류 / 가상 공정·SLAM 관제
15 - 17	Briefit	6인 팀 / AI 담당	입력 정제 / 요약과 정보 보존
18 - 20	Brain MRI	개인 / 전처리·학습·통합 추론	라벨 변환 / 분류·분할 결합
21 - 23	Prompt Generator	개인 / 대화 서버·상태 관리	영역별 이력 / 수정 흐름
24 - 26	Alkkagi.io	개인 / 클라이언트·서버·물리	충돌·검침 보정 / 서버 입력 제한

THING · 텐던 로봇 핸드

기간

2026.07 - 08

팀·역할

6인 팀

모터 제어·기구 통합

사용 도구

DYNAMIXEL / U2D2

모터 점검·제어 스크립트

팀 기술

ROS 2 / MediaPipe

OpenCV / Jetson

Raspberry Pi 5



통합 조립

프로젝트 개요

카메라로 읽은 사람의 손동작을 텐던 로봇손으로 재현합니다. 손동작 인식부터 ROS 2 제어, 모터 구동과 관제까지 연결한 팀 프로젝트입니다.

담당 작업

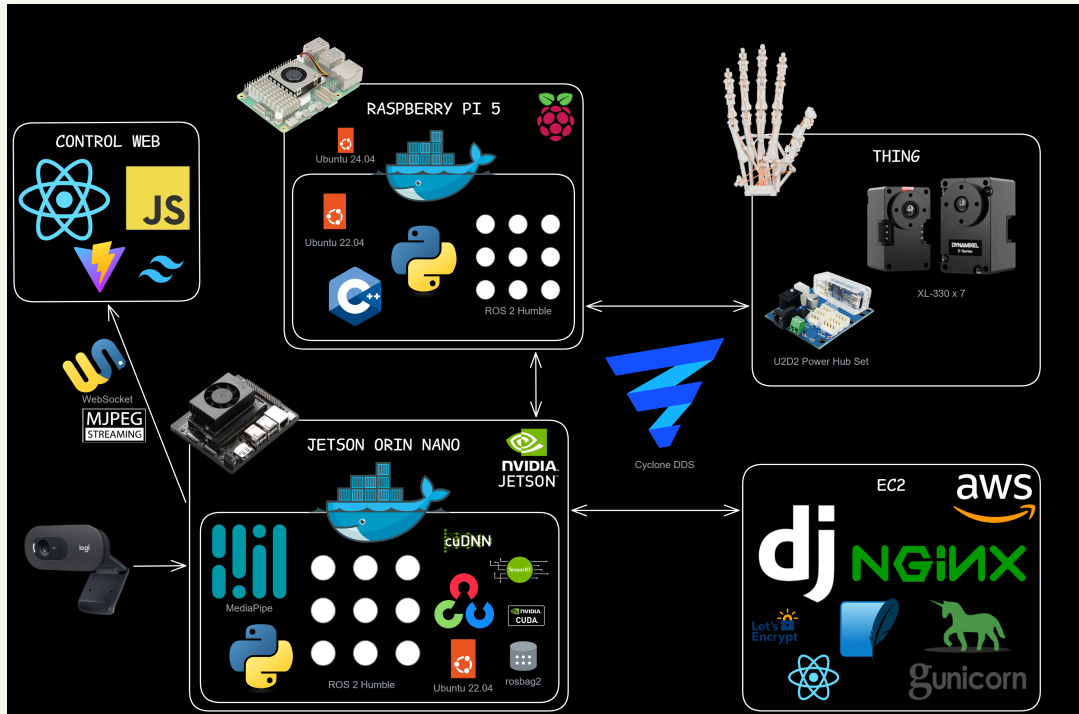
U2D2 통신 환경과 개별·원위치·정지 제어 스크립트를 작성하고, 아크릴 모터 고정부를 제작해 스프링·텐던과 조립했습니다. 소프트웨어의 위치 명령이 실제 손가락 동작으로 이어지는 구동부를 맡았습니다.

협업 범위

손동작 인식, ROS 2 명령 중재·guard, 관제와 데이터 기록은 팀의 협업 결과입니다. 본인 기여는 모터 제어와 기구 통합입니다.

THING · 손 자세를 모터 명령으로 변환

MediaPipe landmark → 7축 목표 → ROS 2 명령 선택·검사 → DYNAMIXEL 구동



인식·ROS 2·guard·운영 드라이버는 팀 구현입니다. 본인은 모터 점검과 기구 통합을 담당했습니다.
DDS는 내부 통신, Jetson → EC2 기록 전송은 HTTPS입니다.

21개 landmark에서 7축 목표로

팀 인식 코드는 관절 굴곡 5축과 엄지 대립·외전 2축을 분리합니다. 굴곡은 관절각을 합성하고, 엄지는 손바닥 폭으로 정규화한 거리와 평면 투영 각도로 계산합니다. deadband·저역통과 필터·프레임별 변화량 제한으로 목표의 흔들림을 줄입니다.

선택된 명령만 구동부로 전달

ROS 2 토픽으로 모방·원격 조작·수동 동작 명령을 분리하고 manager가 제어권을 선택합니다. guard는 상태·시각·축 범위·변화량을 검사합니다. 드라이버가 일반 손가락 축을 엔코더 값으로 바꾸고 엄지 기능 자세를 선택해 Sync Write로 전송합니다.

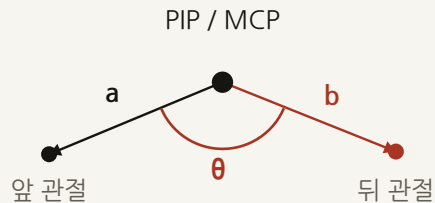
모터 점검 도구와 구동부 조립

U2D2 통신과 개별·원위치·정지 스크립트를 작성했습니다. 이동 명령에서 현재 위치를 Goal Position에 기록한 뒤 Torque ON과 새 목표를 적용했습니다. 이 점검 도구로 구동부를 확인하고 아크릴 고정부·스플·텐던을 통합했습니다.

THING · 관절각 계산과 모터 구동

관절 계산·ROS 제어: 팀 구현 / 본인: 모터 점검·기구 통합 / 수치는 설정값과 계산 예시

손의 세 점으로 굽힘량 계산



$$\theta = \arccos(a \cdot b / (|a| |b|))$$

$$\text{bend} = \text{clip}((180 - \theta) / 125)$$

$$\text{flex} = 0.65 \cdot \text{near} + 0.35 \cdot \text{far}$$

clip: 0~1 / 위 식의 각도는 도 단위 PIP·DIP, 엄지는 MCP·IP를 가중 합산

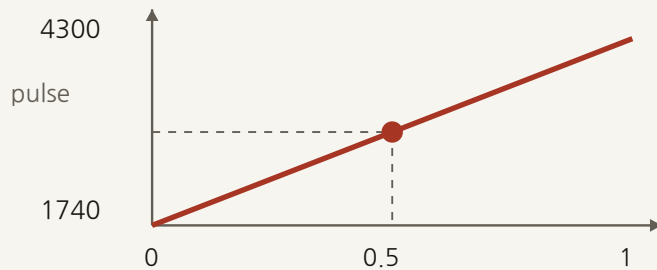
엄지 대립 손끝-손바닥 중심 거리/손바닥 폭을

0.20~1.25에서 역정규화.

엄지 외전 CMC→엄지 끝·검지 MCP를 손바닥 평면에 투영한 각도 10~65°.

21개 영상 landmark 기반 추정값입니다.

굽힘량을 모터의 보정 범위로 변환



3020 pulse

$$1740 + 0.5 \cdot (4300 - 1740)$$

검지 q=0.5의 최종 목표.

실제 송신값은 이동량 제한을 거쳐 접근.

입력 필터 첫 보정 표본으로 초기화.

이후 deadband 0.02 → alpha 0.25 → 표본당 변화 ±0.08.

엄지 네 기능 자세 중 선택. MIMIC 후보는 명령 수신

3회·거리 margin 0.1을 만족해야 전환합니다.

추론과 모터 통신의 주기 분리

추론 도착

발행 20Hz

쓰기 50Hz



$$\text{pps} = \text{profile_velocity} \cdot 0.229 \cdot 4096 / 60$$

$$\text{step} = \max(1, \text{floor}(\text{pps} \cdot \text{speed} \cdot \text{dt}))$$

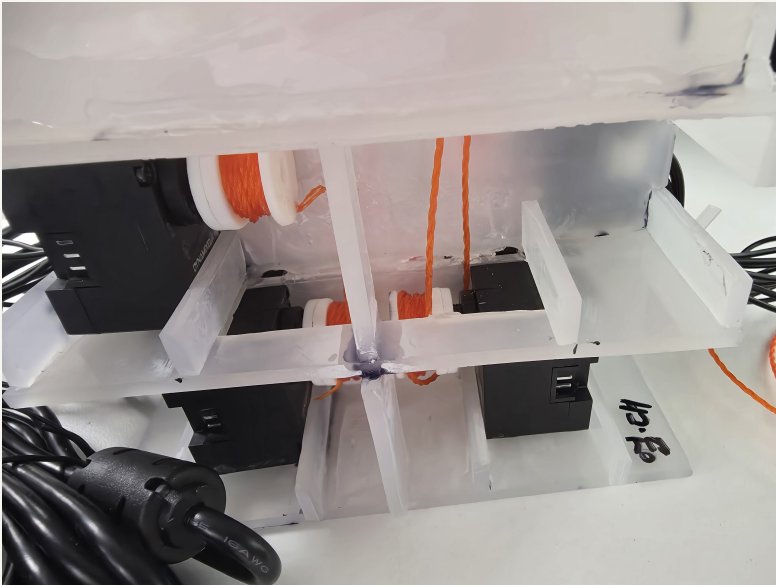
dt ≤ 0.25s / 버스별 Sync Write·읽기 20Hz.

유효 표본 250ms 부재 → 폐기.

300ms 초과 명령 거부 / 실측 주기 보장 없음.

THING · 기구 편차와 제어 조건

같은 모터 위치 명령도 텐던 장력과 권취 방향에 따라 실제 손가락 자세가 달라집니다.



구동부 내부: 모터·스풀·텐던 연결



전원부 모터 고정부

위치 명령과 실제 굽힘

통신 응답이 정상이어도 원하는 자세가 나오지는 별도 확인이 필요합니다. 모터 점검과 실제 기구 동작을 나눠 확인했습니다.

조립 상태를 기록한 이유

모터 고정부를 제작하고 스풀·텐던을 조립했습니다. 체결 방향과 텐던 경로를 사진·작업일지에 남겨 다시 조립할 때 참고했습니다.

이동 명령의 토크 순서

이동 명령에서는 현재 위치 읽기 → 목표값 기록 → 토크 ON 순서를 적용합니다. 이전 목표값으로 갑자기 이동하는 것을 막기 위한 점검 코드입니다.

원위치와 정지

다회전 위치에서 가장 가까운 중앙각 좌표를 선택합니다. 키보드로 개별·전체 모터의 토크를 끄는 Torque OFF 경로도 마련했습니다.

원위치 복귀와 축별 보정

중앙각 복귀만으로 손가락의 가동 범위가 보정되지는 않습니다. 당시 일지에서 축별 끝점 보정과 최대 가동 범위의 간섭 검증을 후속 과제로 남겼습니다.

THING · 실물 동작과 결과



캔 파지 시연

수상

SSAFY 공통 프로젝트 우수상

2026.08.10 / THING

[상장 보기](#)

팀 시연

손동작 모방과 손가락 순차 동작, 캔·부드러운 물체 파지를 시연했습니다. 인식과 제어 명령이 조립한 구동부를 거쳐 실제 손가락 동작으로 이어졌습니다.

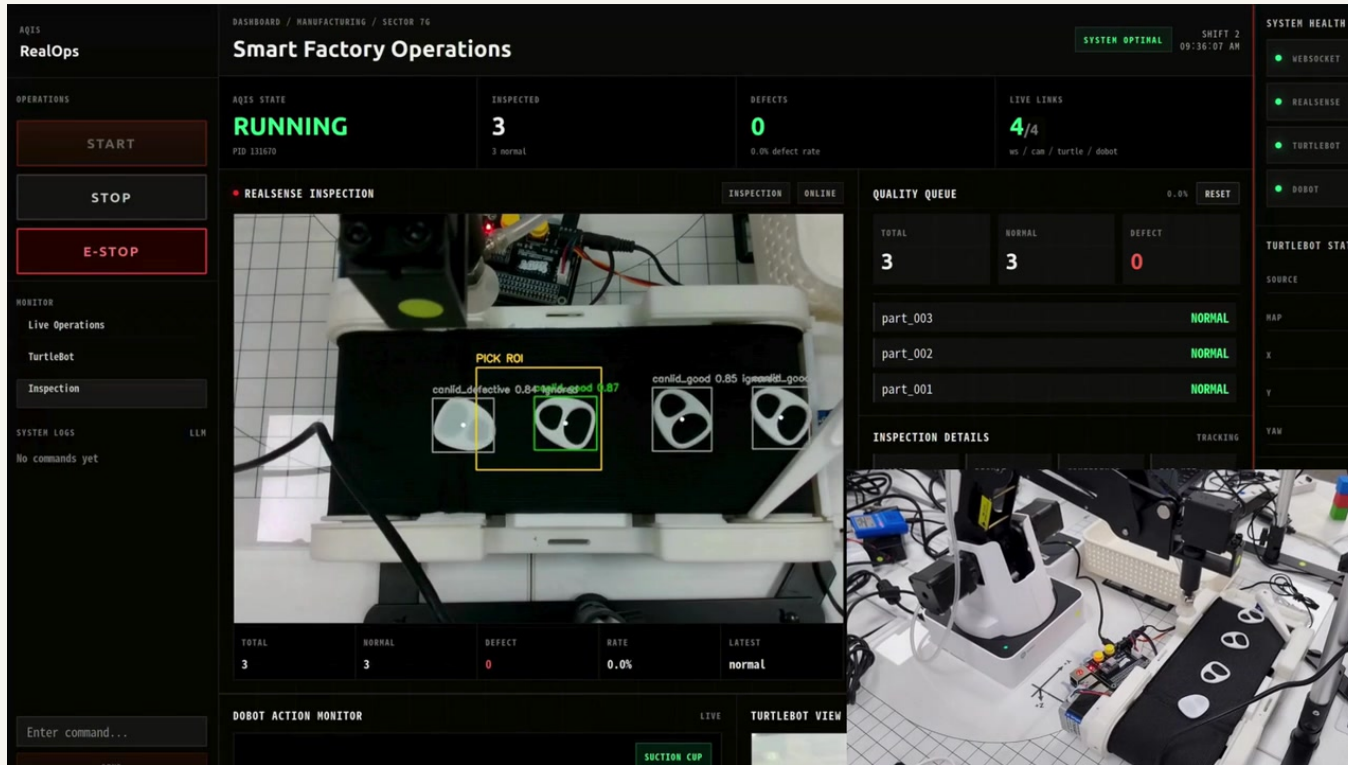
파지 시연과 추가 시험

손동작에 따라 실제 구동부가 움직이는 것을 확인했습니다. 물체별 파지 성공률과 재조립 후 반복성을 비교하려면 장력·가동 범위와 시험 조건을 고정한 반복 측정이 필요합니다.

보정 전후 편차·반복 파지 성능의 정량 결과는 공개 기록에서 확인되지 않았습니다.

AQIS · 스마트 팩토리

실제 검사·Dobot 분류, RoboDK 가상 공정, TurtleBot SLAM·로봇 상태 관제를 연결



RealOps · 실제 장비 시연

협업 팀원은 모델 학습·RoboDK·CAD·시뮬레이션을 담당했습니다.

2026.05 기획 / 06 본 개발

2인 팀 · 팀장

Full-stack & Robot Integration

프로젝트 개요

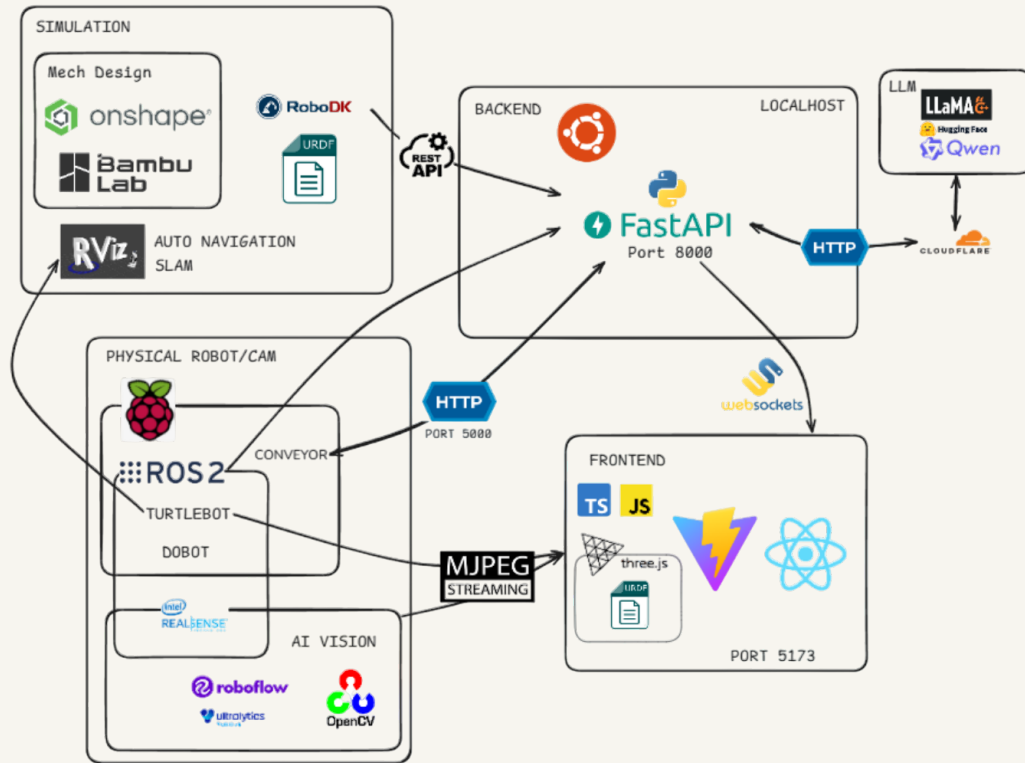
카메라 검사에 따라 Dobot이 제품을 분류합니다.
RoboDK 가상 공정과 TurtleBot SLAM·상태 관제를
함께 만들고, 공통 서버로 웹 화면과 연결했습니다.

직접 맡은 일

React 관제와 FastAPI·WebSocket 서버, ROS 2
연결, 장비 adapter와 집기 시퀀스를 구현했습니다.
서버에서 검출 이벤트를 장비 명령으로 연결했습니다.

AQIS · 검출을 공정 제어로 연결

YOLO·깊이 카메라 → ROS 2 검출 이벤트 → FastAPI 상태 관리 → 장비 명령·WebSocket 관제



본인: 관제·서버·ROS 연결·장비 adapter·집기. 비전·학습: 팀 구현. TurtleBot 자동 임무는 확장 설계이며 SLAM·카메라·상태 관제를 구현했습니다.

영상의 검출 중심을 공간 좌표로

팀 비전 노드는 YOLO 검출 중심이 집기 ROI 안에 있는 후보를 남깁니다. 중심 주변 유효 깊이의 중앙값과 카메라 내부 파라미터로 픽셀을 3D 좌표로 역투영합니다. 라벨 누적과 동일 라벨·위치 이벤트의 재발행 제한을 거쳐 결과를 ROS 토픽에 보냅니다.

검출 형식을 통일해 상태를 갱신

직접 구현한 서버는 ROS 콜백의 JSON을 개별 검출로 펼치고 정상·불량 라벨을 공통 형식으로 정규화합니다. 중복을 거른 뒤 통계와 공정 상태를 갱신하고, asyncio 루프의 WebSocket으로 React 관제에 전달합니다.

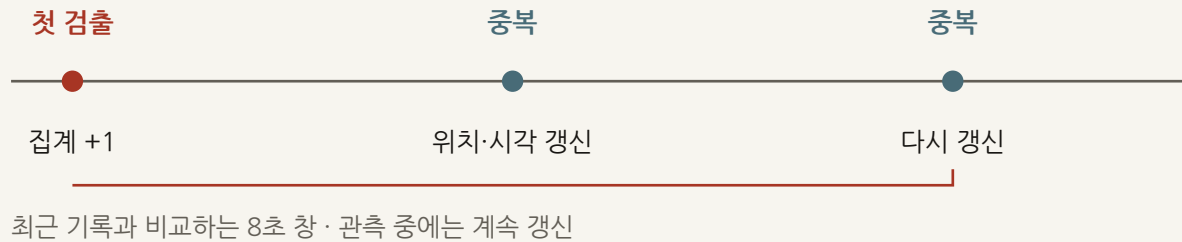
장비 연결부와 집기 좌표의 분리

초기 계획의 실물 사용은 마지막 3일이었습니다. 공통 API 아래 Mock·실장비 adapter를 두어 관제 흐름을 먼저 연결했습니다. 정지 요청 뒤 갱신한 카메라 X/Y는 affine 계수·단위·오프셋을 적용해 Dobot 집기 목표로 변환합니다.

AQIS · 중복 검출과 집기 좌표 처리

중복 집계와 집기 좌표 갱신을 분리 / 좌표 연결·서버·관제: 본인 구현 / 비전 역투영: 팀 구현

반복 검출은 한 번만 집계



같은 key: label·result·is_defect

중복 조건: 명시적 ID 일치 / IoU≥0.5 / 중심 거리≤70px 중 하나.

같은 불량에서 양쪽 bbox·center가 모두 없어도 중복입니다.

집기 대기(pending)는 중복 판정보다 먼저 처리



RUNNING + pending

준비 시각 이후·이벤트 나이≤3초의 불량 중 깊이 있는 후보 우선.

후보가 있을 때 집기 요청·pending 해제.

timestamp 누락은 허용하며 최초 물체 ID를 고정하지 않습니다.

카메라 좌표를 Dobot 작업 좌표로 변환

$$X=(u-cx)*d/fx; Y=(v-cy)*d/fy$$

$$x_{mm}=1000*(a11*X+a12*Y+tx)+ox$$

검출 상자 중심 → 깊이 격자 → 유효 깊이 중앙값 d(m).
로봇 y도 affine, z는 고정 높이 또는 X/Y affine. 깊이 누락은 고정 좌표.

재개 조건 재개 설정 ON + 종료 코드 0.

정지 응답 실패 시에도 자동 차단하지 않습니다. 실제 파지 성공은 센서로 확인하지 않습니다.

AQIS · 좌표 갱신과 집기 제어

이동 중의 최초 좌표를 계속 쓰지 않고, 정지 요청 뒤 후속 검출로 집기 위치를 갱신합니다.



집계 중복과 명령 중복 처리

같은 대상이 여러 프레임에 나타나므로 객체 ID와 영역 겹침·중심 거리로 중복 집계를 억제합니다. 집기 작업 중에는 명령의 중복 실행도 막습니다.

정지 상태와 오래된 입력 처리

관제가 정지되면 검출에 따른 통계 갱신과 집기를 시작하지 않습니다. timestamp가 있는 검출은 준비 시점 이전이거나 허용 연령을 넘으면 제외합니다.

명령 완료와 실제 파지 성공

설정 시간 대기와 스크립트 종료 코드는 실제 정지·파지 성공의 센서 확인이 아닙니다. 자동 흐름은 정지 명령의 실패 응답으로 다음 단계를 차단하지 않습니다.

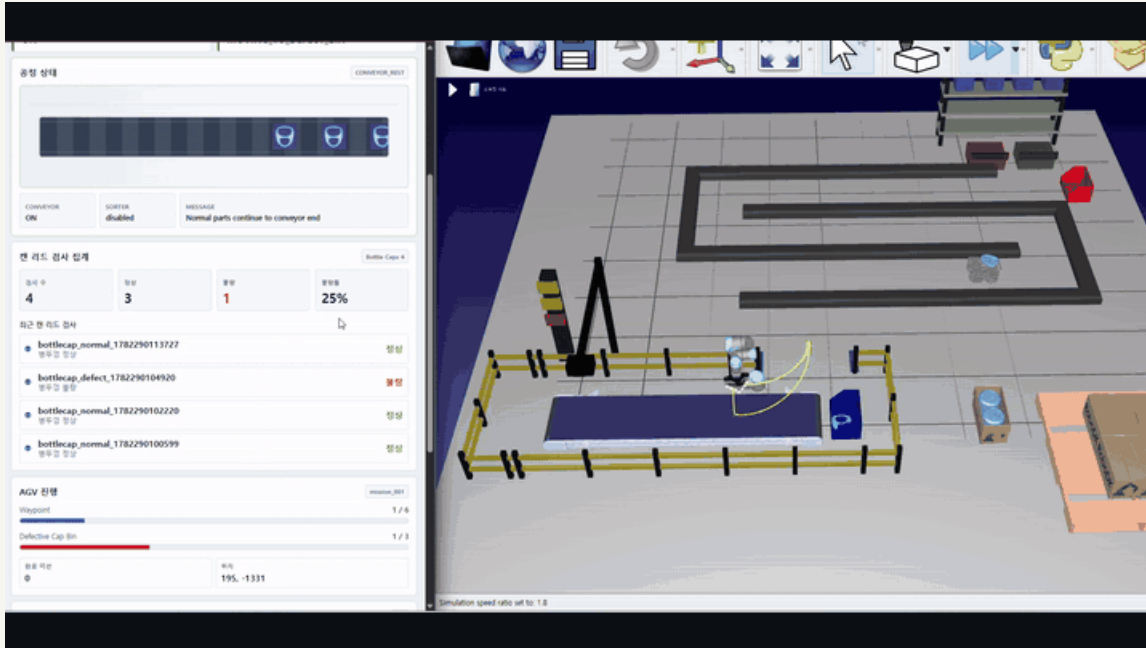
입력 누락과 남은 확인 사항

timestamp가 없으면 신선도 검사를 통과하고, 깊이가 없으면 고정 좌표를 사용합니다. 좌표 변환 기대값 불일치와 실물 반복 성능·통신 복구는 추가 확인이 필요합니다.

중복·정지 상태·시각 조건: 원본 테스트와 후속 Mock 재현으로 확인. 실물 성공률·사이클 시간의 측정 결과는 없습니다.

AQIS · RoboDK 디지털 트윈

RoboDK 공정·Simulation Dashboard: 팀원 담당 / 본인: 서버·명령·이벤트 연결과 실제 장비 통합



공개 시연 · Simulation Dashboard와 RoboDK 공장 장면

검사·집기·운반 시뮬레이션

하드웨어 준비 전, 검사·집기·운반 순서와 웹 인터페이스를 먼저 연결했습니다.

컨베이어

$x += \text{speed} \times dt$ / 이름·위치로 검출

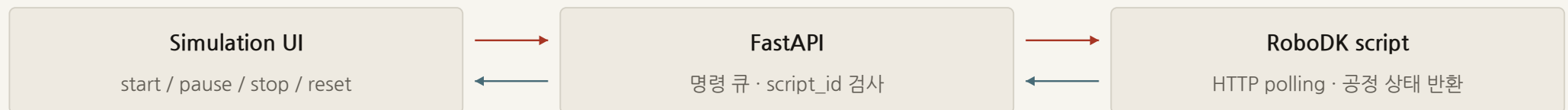


UR5 집기

Home → Pick → Mid → Place / MoveJ

객체의 부모를 그리퍼에서 TurtleBot으로 바꿔 적재를 표현합니다.
AGV는 nav1~nav6 프레임 사이를 보간해 이동합니다.

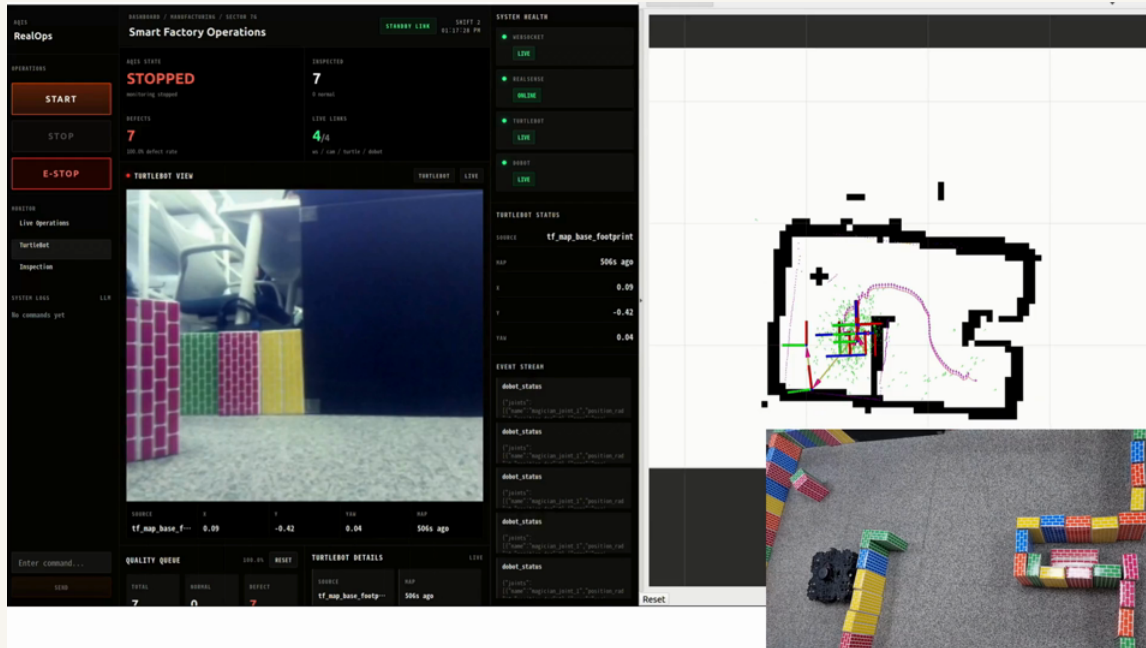
명령 전달과 상태 반환



공정·기구학 시뮬레이션입니다. 접촉력·파지 성공을 계산하거나 실제 Dobot 관절을 RoboDK에 동기화하지는 않습니다.

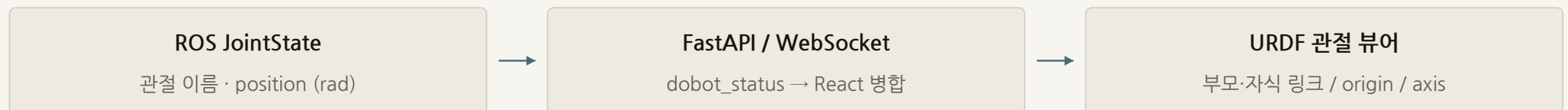
AQIS · SLAM · 로봇 상태 관제

외부 ROS 패키지·SLAM 시연은 팀 결과 / 본인: ROS bridge·FastAPI·RealOps·Dobot 관절 뷰어



공개 시연 · 왼쪽 RealOps, 오른쪽 RViz 지도·실물 TurtleBot

관절값으로 URDF 자세 갱신



뷰어 각도 = $\text{position_rad} \times \text{multiplier} + \text{offset (mimic)}$. ROS_ENABLED 환경에서 실시간 상태를 반영합니다.

지도와 로봇 위치 수집

`/map → map_update`
OccupancyGrid → PNG·해상도·원점.
지도 이벤트의 2초 이내 재발행 억제.

TF → AMCL → odom 대체 입력

`map → base_footprint/base_link`를 0.1초 간격으로 조회. 수신 콜백은 최신 TF(0.8초)·AMCL(2.5초)를 우선합니다.

RealOps 화면

카메라·위치·지도 갱신 시각을 표시하고 지도 PNG 자체는 그리지 않습니다. odom은 map 좌표로 재변환하지 않아 출처를 함께 표시합니다. Nav2 goal 자동 전송은 미구현입니다.

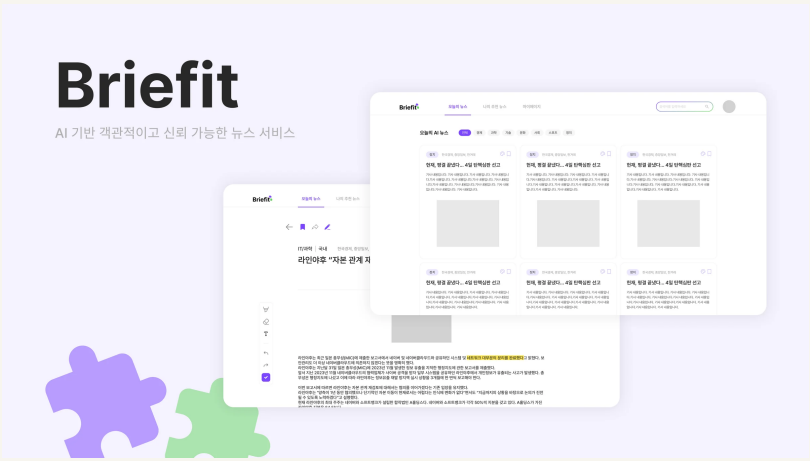
Briefit · 뉴스 수집과 요약

여러 매체의 뉴스를 모아 요약을 제공하는 서비스 / 2025년 본인 AI 구현

기간
2025.05 - 09

팀·역할
6인 팀 / AI 2인
본인: AI 담당

사용 기술
Python / Crawl4AI
BeautifulSoup
Transformers / KoBART



서비스 UI · 팀 결과

직접 구현한 범위

기사 본문 정제와 배치 간 URL 중복 방지, KoBART 학습·생성·평가 스크립트를 구현했습니다. 긴 기사의 부분 요약·재요약과 생성문 후처리를 맡았습니다.

수집·생성·후처리 분리

수집, 모델 생성, 후처리를 나눠 기사 중복과 요약 누락·반복을 각각 점검할 수 있게 했습니다.

프로젝트 수상

2025 IT대학 소프트웨어 공모전 금상

2025.08.18 / Briefit

[상장 보기](#)

제15회 숭실 캡스톤디자인 경진대회 장려상

2025.10.01 / Briefit

[상장 보기](#)

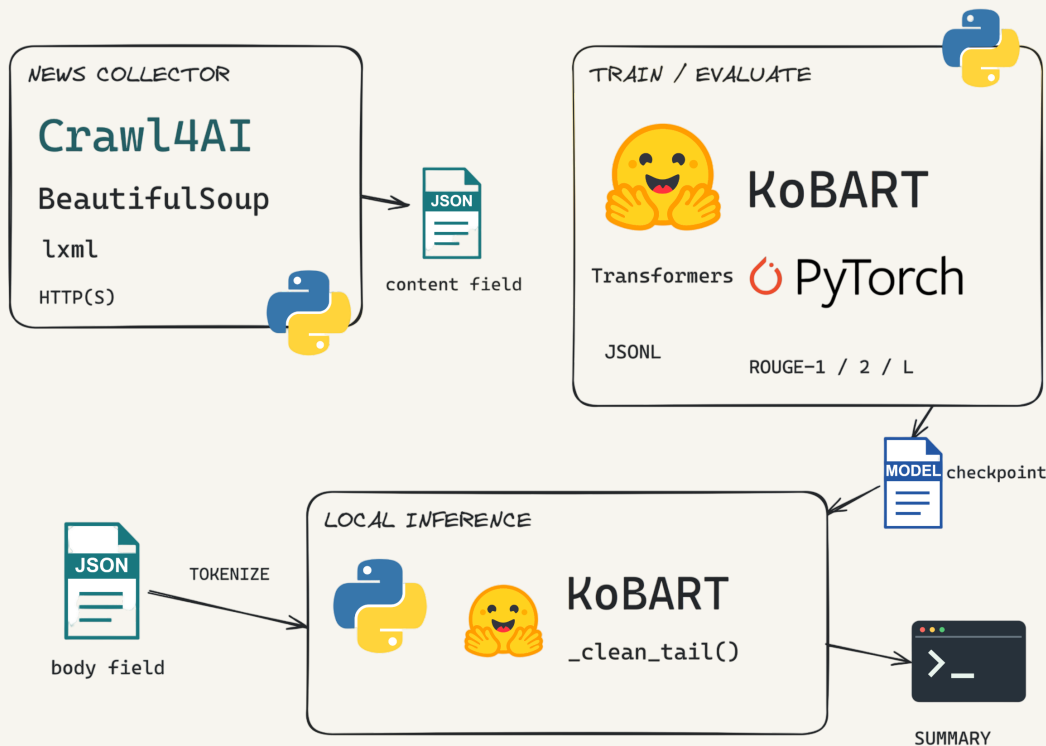
2025 IT 프로젝트 프로리그 장려상

2025.11.22 / Briefit

[상장 보기](#)

Briefit · KoBART 학습과 요약 생성

기사·기준 요약으로 학습 구성 → 기사에서 요약 토큰 생성 → 긴 입력 재요약·출력 정리



2025년 본인 작업: 본문 정제·실행 내 URL 집합 공유, KoBART 학습·생성·평가 코드. 학습 완료 로그·ROUGE 점수와 후처리 전후 품질 비교는 확인되지 않았습니다.

기사와 기준 요약을 학습 쌍으로

사전학습 KoBART의 인코더는 기사 문맥을 표현하고 디코더는 요약 토큰을 생성합니다. 기사 text를 입력 토큰으로, 기준 summary를 정답 labels로 변환해 Seq2SeqTrainer에 전달합니다. 본인은 토큰화·학습 설정·평가 스크립트를 구성했습니다.

긴 입력은 부분 요약 뒤 재요약

생성 시에는 기사만 입력하고 beam search로 요약 후보를 탐색합니다. 긴 입력은 문단을 묶어 부분 요약한 뒤 결과를 이어 붙여 재요약합니다. 입력을 나누더라도 긴 문단과 재요약 입력은 토큰 제한으로 잘릴 수 있어 전체 문맥 보존을 보장하지 않습니다.

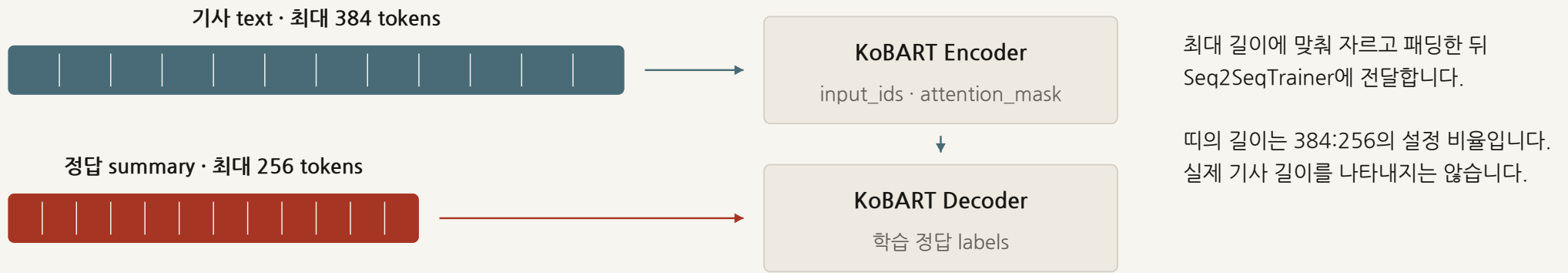
모델 생성과 규칙 후처리의 분리

별도 함수에서 생성문 끝의 반복·짧은 문장을 제거합니다. 정상 문장도 삭제될 수 있어 반복 감소와 정보 손실을 함께 확인해야 합니다. ROUGE는 기준 요약과 후처리 전 생성문을 비교하며, 최종 서비스 출력의 후처리 효과를 평가하지는 않습니다.

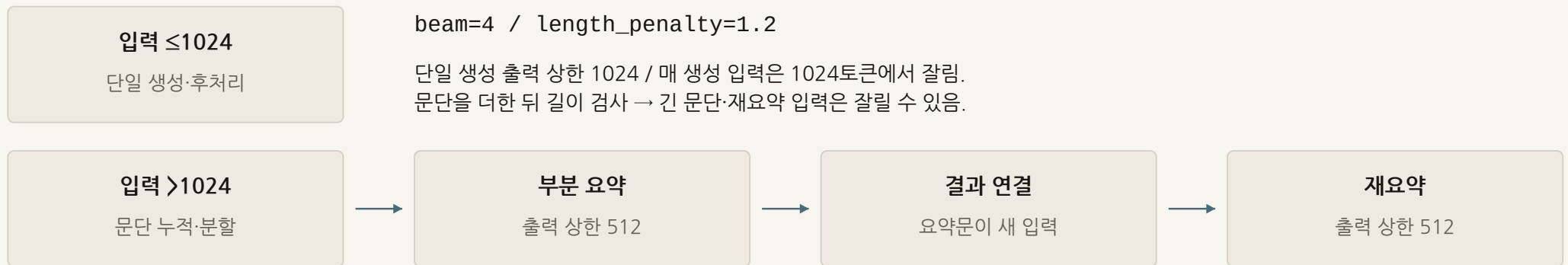
Briefit · 학습 입력과 긴 기사 요약

기사·정답을 학습 쌍으로 구성하고 긴 기사는 부분 요약을 거쳐 재요약 / 본인 2025년 KoBART 소스

기사와 정답을 따로 토큰화



긴 기사는 나눠 요약하고 다시 연결

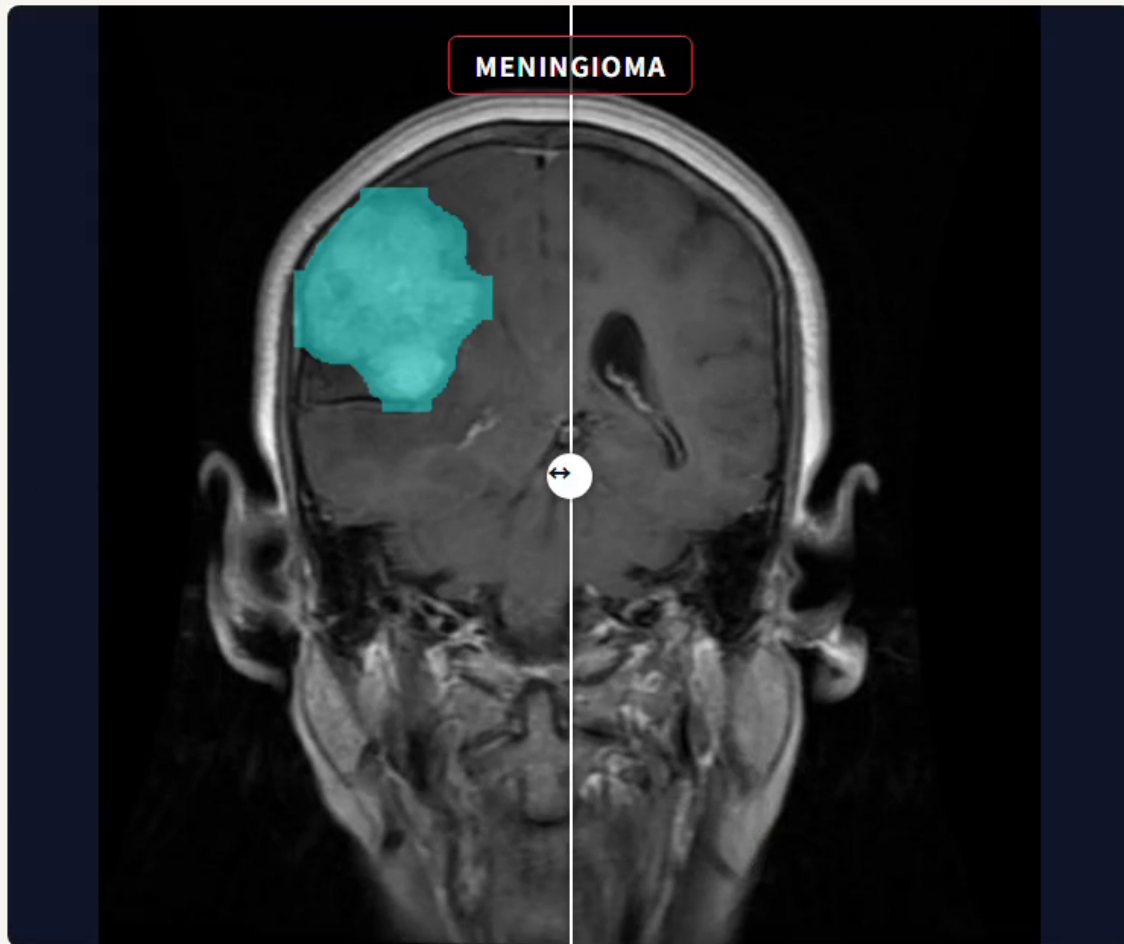


각 생성의 clean_tail: 짧은 끝문장·직전 반복 제거

평가 경로: 기사 1024 → raw 생성 128 → ROUGE(reference). 부분·재요약/후처리를 거치지 않습니다.
후처리는 각 부분 요약에도 적용돼 정상 정보를 지울 수 있으며, 학습 완료·실측 점수는 확인되지 않았습니다.

Brain MRI · 종양 분류와 영역 분할

개인 프로젝트 / 2026.02 - 04 / Python · PyTorch · YOLO11 · OpenCV



분류·분할 통합 추론 · 원본 데모

프로젝트 개요

MRI의 종양 유형을 분류하고 영역을 분할하는 연구·학습용 프로젝트입니다. 분류 범주와 분할 위치를 같은 이미지에서 비교할 수 있도록 구성했습니다.

직접 구현한 범위

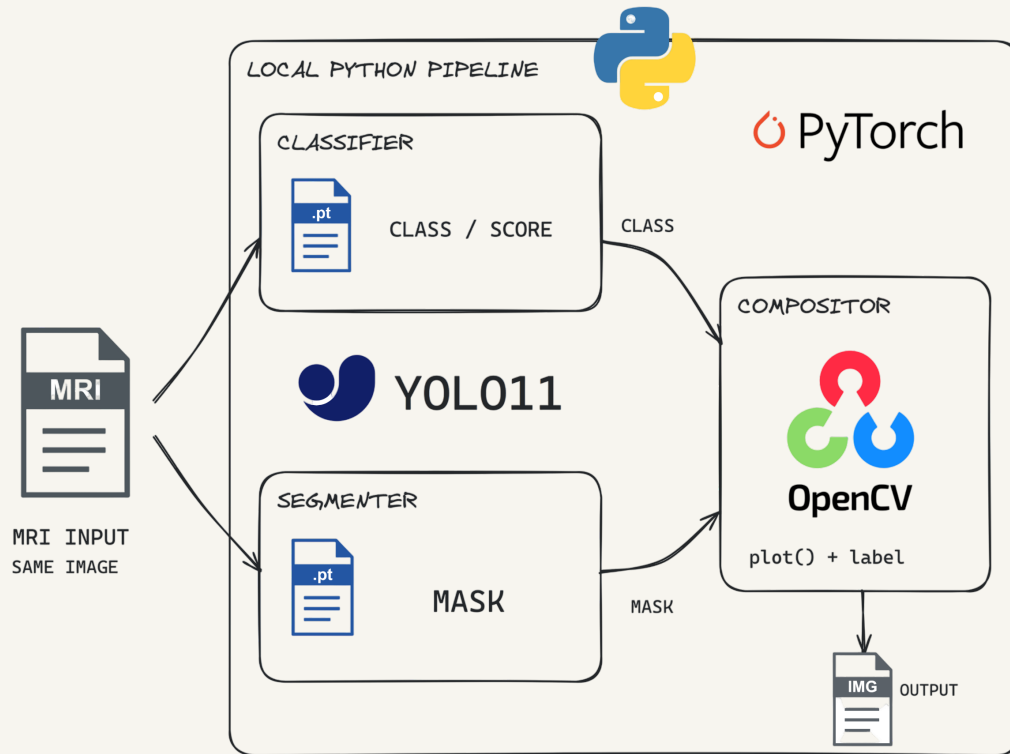
BRISC 이미지·마스크를 학습 형식으로 변환하고, 두 YOLO11 모델의 학습과 통합 추론 코드를 작성했습니다. 데이터 전처리부터 예측 시각화까지 연결했습니다.

학습 입력: 분류용 이미지 폴더 / 분할용 polygon label

출력: 분류 범주와 분할 영역의 통합 시각화

Brain MRI · 두 모델의 분류와 분할

이미지·마스크를 학습 형식으로 변환 → 두 모델에 독립 입력 → 범주와 분할 영역 시각화



평가 범위 학습에서 test를 val에 연결해 별도 독립 평가가 필요합니다. BRISC의 환자 단위 독립성은 확인할 수 없고 비종양 범주가 정상만 뜻하지는 않습니다. 임상 진단 검증은 완료하지 않았습니다.

유형과 위치를 서로 다른 모델로

YOLO11 분류 모델은 이미지 전체의 범주를, 분할 모델은 영역의 위치와 형태를 예측합니다. 클래스별 이미지 폴더와 이미지·polygon label 쌍으로 학습 입력을 나누고, 사전학습 cls·seg 모델을 각각 불러와 별도 가중치로 추론하도록 구성했습니다.

픽셀 마스크를 정규화 좌표로

마스크를 이진화하고 closing·opening으로 정리한 뒤 외부 윤곽을 추출합니다. 면적·점 수 조건을 통과한 꼭짓점의 x/y를 너비/높이로 나눠 polygon label로 저장합니다. 이 과정에서 작은 병변이나 내부 구멍이 사라질 수 있어 원본과 대조가 필요합니다.

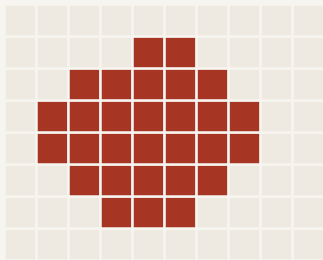
같은 MRI의 두 출력을 함께 표시

같은 MRI를 두 모델에 각각 입력합니다. 분류의 최상위 범주·점수를 읽고 분할 마스크가 그려진 이미지 위에 표시합니다. 비종양 분류도 분할 추론을 차단하지 않으며, 두 결과가 다를 때 자동으로 판정을 보정하는 구조는 아닙니다.

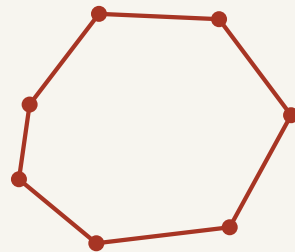
Brain MRI · 마스크의 YOLO 라벨 변환

mask > 1 → 5×5 closing·opening → 외부 윤곽 → 정규화 좌표 / 아래 도형은 표현 변환 개념도

마스크 정제와 윤곽 추출



이진 마스크



외부 윤곽점



points >= 3

area >= 0.001*W*H

RETR_EXTERNAL / CHAIN_APPROX_SIMPLE
직선의 중간 점을 줄이고 작은 윤곽을 제거.

고정 5×5 커널: 해상도에 따라 상대 정제 범위 변화. 외부 윤곽·면적 필터는 내부 구멍·작은 실제 병변을 지울 수 있음.

정규화 라벨 저장과 두 모델의 추론

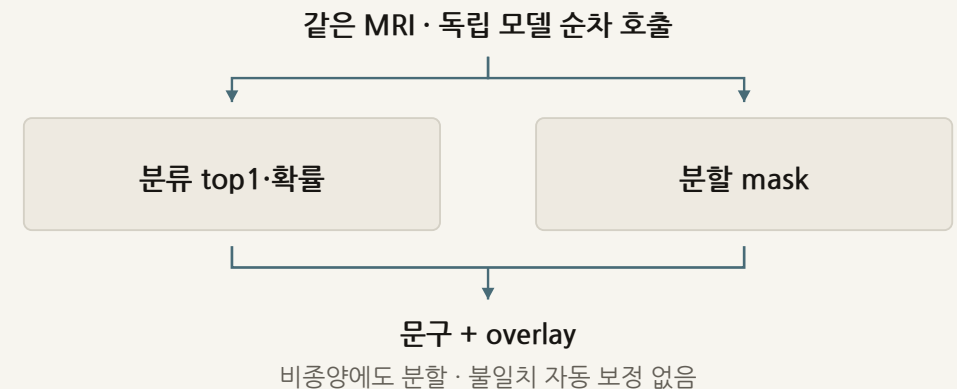
$x_norm = x/W$ $y_norm = y/H$

class_id x1 y1 x2 y2 ...

YOLO polygon label

class_id는 파일명의 종양 코드입니다.

같은 basename의 txt에 소수점 여섯 자리로 저장합니다.



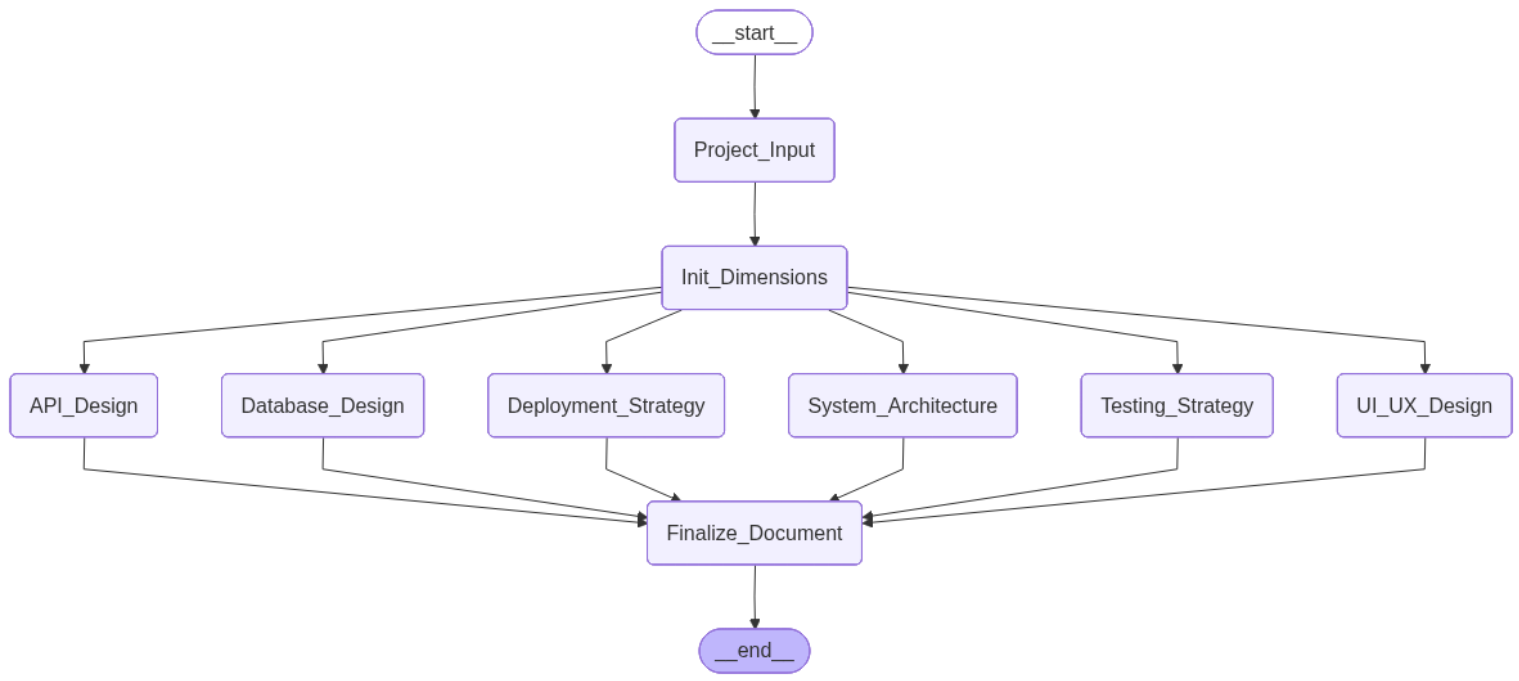
Prompt Generator · 프로젝트 설계 도우미

아이디어를 영역별 질문으로 구체화하고 결과를 하나의 문서로 모으는 도구

기간
2026

팀·역할
개인 프로젝트
대화 서버·상태 관리
웹 UI

사용 기술
Python / FastAPI
WebSocket / LangChain
Solar Pro



초기 설계 흐름 · 원본

프로젝트 개요

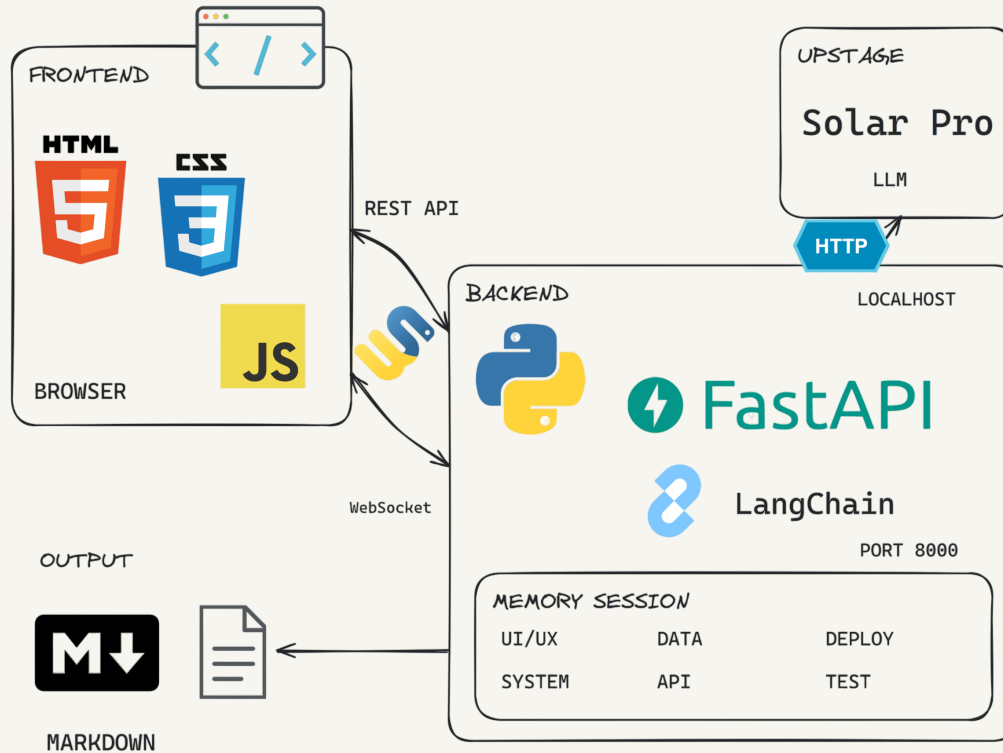
프로젝트 아이디어를 입력하면 화면·API·데이터 등 영역별 질문으로 요구사항을 구체화합니다. 각 영역의 결과를 하나의 설계 문서로 모읍니다.

직접 맡은 일

대화 서버와 웹 UI를 연결하고, 질문·답변에서 결과 문서 생성까지 진행 상태를 관리했습니다. 완료된 영역을 추가 대화로 수정하는 흐름도 구현했습니다.

Prompt Generator · 영역별 대화 관리

프로젝트 입력 → 영역별 지침·이력 → Solar Pro 응답 → 상태 갱신 → 설계 문서 작성



검증 범위 완료 조건은 생성 품질의 판정이 아닙니다. 처리 오류의 자동 재시도는 없고, 메모리 세션은 연결 종료 시 삭제됩니다. 재접속 복구와 요구사항 반영률·생성 품질의 평가 결과는 없습니다.

영역별 이력으로 다음 질문 구성

UI·API·DB·구조·테스트·배포의 여섯 영역을 나눕니다. 호출마다 영역 지침·프로젝트 설명·해당 대화 이력을 LangChain 메시지로 묶어 Solar Pro에 전달합니다. 첫 질문은 `asyncio.gather`로 병렬 실행하고 FastAPI가 WebSocket으로 응답을 전송합니다.

응답과 서버 상태를 함께 관리

영역별 라운드·이력·결과를 저장합니다. 처리 라운드가 3 이상이고 응답에 `[GENERATE_PROMPT]` 태그가 있으면 완료로 전환하며 추가 대화로 수정할 수 있습니다. 입력·응답은 호출 성공 뒤 기록하고, 처리 대상 오류는 라운드를 되돌리고 대기 상태로 전환합니다.

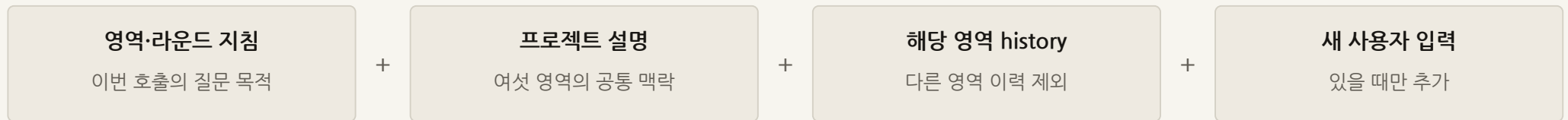
영역별 결과에서 최종 문서로

결과가 저장된 영역의 프롬프트와 프로젝트 설명을 모아 별도 LLM 호출로 설계 문서를 작성합니다. 영역 하나의 결과만 있어도 문서를 만들 수 있습니다. 웹 UI와 대화 상태 관리, 최종 문서 생성까지 직접 구현했습니다.

Prompt Generator · 문맥 분리와 상태 전이

영역별 대화 이력을 따로 관리하고, 저장된 결과를 모아 설계 문서 작성

영역별 LLM 호출에 넣는 메시지

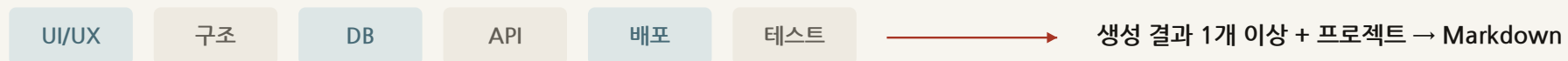


첫 질문: asyncio.gather 병렬 시작 / 동기 LLM invoke: to_thread로 이벤트 루프와 분리

응답에 따른 상태 변경



결과가 있는 영역으로 문서 작성



색은 결과가 있는 영역의 예시입니다. 수정이 실패해도 이전 generated_prompt는 남습니다. 세션은 연결 종료 시 삭제됩니다.

Alkkagi.io · 실시간 알까기

여러 플레이어가 같은 보드에서 돌을 밀어내며 대전하는 웹 게임

기간

2026.03 - 04

팀·역할

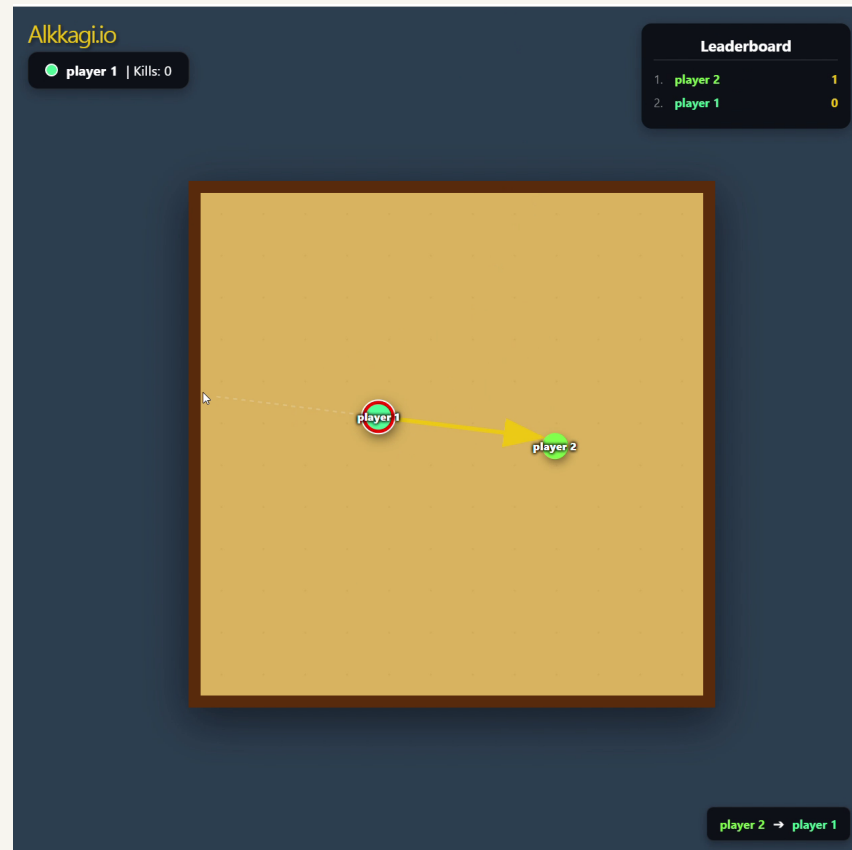
개인 프로젝트

웹 UI·서버·물리 계산

사용 기술

React / TypeScript

Node.js / Socket.IO



실제 플레이

프로젝트 개요

같은 보드에 접속한 플레이어가 돌을 밀어내며 대전하는 웹 게임입니다. 돌의 움직임과 충돌 결과를 실시간으로 공유합니다.

직접 맡은 일

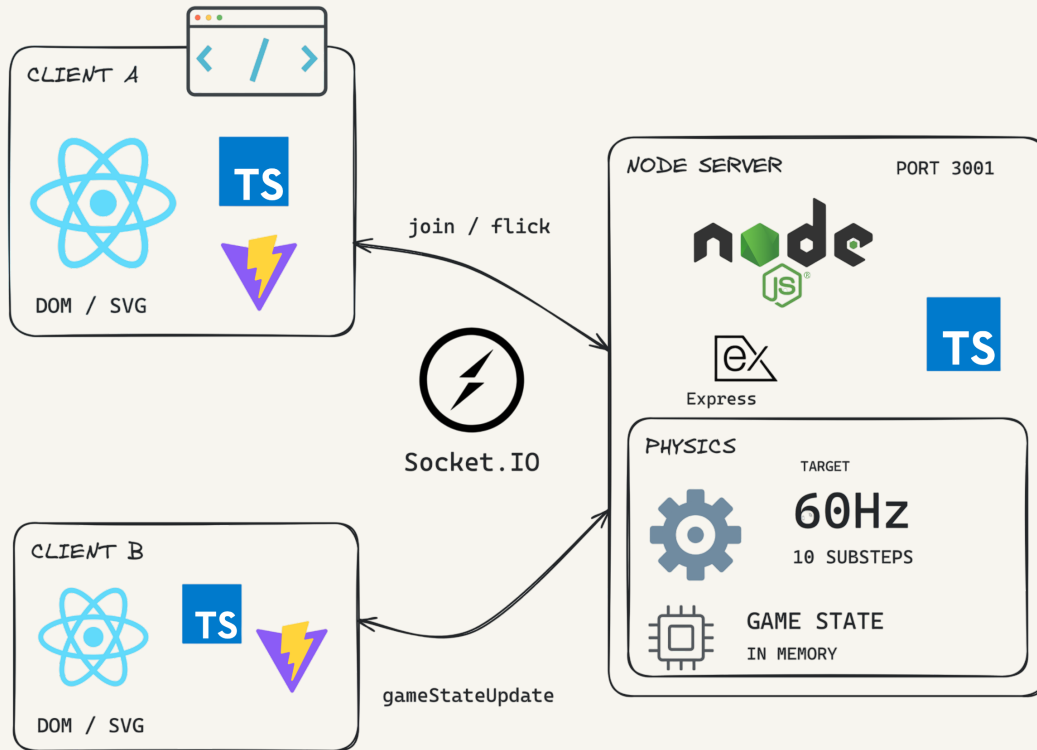
React 조준 UI와 Socket.IO 통신, 서버의 게임 상태·물리 계산을 구현했습니다. 충돌·마찰·검침 보정은 TypeScript로 작성했습니다.

게임 규칙의 실행 위치

클라이언트는 발사 입력을 보내고 서버가 기준 상태를 계산합니다. 입력 제한과 물리 규칙을 모든 플레이어에게 같은 기준으로 적용합니다.

Alkkagi.io · 서버 물리와 상태 동기화

드래그 벡터 → Socket.IO 발사 입력 → 이동·마찰·충돌 계산 → 모든 클라이언트에 상태 전송



개인 구현·검증 범위 웹 UI·통신·물리 함수를 직접 작성했습니다. 60Hz는 갱신 설정이며 부하·지속 프레임률의 실측값은 없습니다. 연결 종료 시 플레이어를 정리하며 재접속·서버 재시작 복구는 없습니다.

서버에서 입력 검증과 상태 계산

React의 드래그를 속도 벡터로 바꿔 flick 이벤트로 보냅니다. 서버는 발사 간격·최대 속도를 제한하고 질량을 반영해 속도를 정합합니다. 위치·속도·반지름·질량은 서버 메모리에 두며, 갱신 후 gameStateUpdate를 배포해 클라이언트가 화면을 그립니다.

겹친 위치와 충돌 속도를 분리

중심 거리와 반지름 합으로 겹침을 찾고, 겹친 거리는 두 돌에 절반씩 나눠 위치를 보정합니다. 충돌 방향으로 이미 멀어지면 충격량을 더하지 않습니다. 접근하는 돌은 상대 속도·반발계수·역질량으로 충격량을 계산해 각각의 속도를 갱신합니다.

소단계에서도 감속 비율 유지

한 번의 갱신을 10개 소단계로 나눠 이동·충돌을 계산합니다. 마찰 계수에 소단계 비율을 지수로 적용해 분할 횟수만큼 감속이 중복되지 않게 했습니다. 보드 이탈 시 충돌 기록으로 점수·재배치를 처리하고 돌의 반지름·질량을 갱신합니다.

Alkkagi.io · 입력 제한과 충돌 반응

서버가 입력을 제한하고 위치·마찰·충격량을 계산한 뒤 전체 상태 배포 / 도형과 수치는 계산 예시

입력 크기를 제한하고 질량으로 나누기

입력 (300,400)



크기 500

상한 (162,216)



$0.45 \times \text{보드 } 600 = 270$

속도 (108,144)



질량 1.5로 나눔

서버 입력 조건

접속자의 돌 확인 · 500ms 재입력 거절

질량 = $1 + 0.05 \times \text{kills}$

속도 단위는 서버 갱신 기준

10개 소단계에서 감속 비율 유지



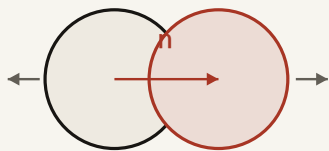
$p += 0.1 \cdot v; v *= 0.8^{**}0.1$

매 단계: 축 속도 $|v| < 0.1 \rightarrow 0$ / 충돌 / 득점 후 재배치.

10회 뒤 상태 전송. 충돌·정지 절삭이 없으면 총 0.8배 감쇠.

타이머는 1000/60ms이며 실제 경과 시간으로 보정하지 않습니다.

겹침을 보정한 뒤 충격량 계산



overlap/2

양쪽 위치를 절반씩 보정

일반 경우: $0 < D < r1 + r2$

$vn = \text{dot}(v2 - v1, n); vn > 0: \text{skip impulse}$

$j = -(1 + 0.7) \cdot vn / (1/m1 + 1/m2)$

$v1 -= (j/m1) \cdot n; v2 += (j/m2) \cdot n$

연락처

Se Min Kong

공세민 · 소프트웨어 개발자

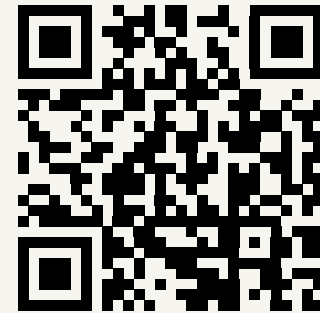
이메일

semin1224@gmail.com

GitHub

github.com/SeMinKong

웹 포트폴리오



seminkong.github.io/SeMinKong_Web/

[프로젝트 영상과 상세 설명](#)

[이력서와 상장 전시](#)